

LEMMON AGENTES - Manual do Sistema **Versão atual:** v1.26 **Última
atualizaçãõ: 2026-05-07 **Mantido por: Calebe Alves / Lemmon Produções > Este
é o documento de referência viva do sistema Lemmon Agentes. Sempre que uma função
nova for implementada ou um ícono fechar, este manual deve ser atualizado e uma nova
versão de PDF gerada em `docs/releases/`. --- ## Histórico de versões >

Convenções: versões mais novas no topo. Cada release lista o que mudou em
relação à anterior, mantendo histórico completo. ### v1.26 - 2026-05-07 **T98 (hotfix)
+ T97: piso watchdog 60s → 180s + Otto modo_visual 'auto'. - **T98 - watchdog piso
corrigido: `max(60, ceil(â†` `max(180, ceil)` em `useReuniao.ts` e `useChat.ts`. Com o piso
anterior, Otto (fallback 20s) disparava em `max(60, 60) = 60s` menos que 1 ciclo normal.
Com piso 180s: mínimo 3 min garantidos para qualquer agente. - **T98 - mensagem
correta: `Math.round(timeoutMs / 60000)` já calculava o valor real; com o piso certo exibe
"3min" para Otto e o tempo proporcional para os demais. - **T97 - Otto `modo\_visual`: tipo
atualizado de `completo` | `resumido` | `minimo` para `completo` | `resumo` | `auto` alinhado
com `core/validador.py` (backend já usava esses valores). Default muda de `completo` para
`auto` (Otto escolhe o nível de detalhe baseado na complexidade do briefing). - **T97 -
ConfigSidebar: botões "Completo / Resumo / Auto (IA decide)". - **T97 - bug silencioso
corrigido: `api/ws\_chat.py` passava "resumido" em fast-track; `validar\_modos\_visual()`
rejeitaria esse valor. Corrigido para "resumo". - **Manual: - 2.1 (tabela modo_visual),
3.2 + 4.17 (piso watchdog). --- ### v1.25 - 2026-05-07 **FASE 6 - T95: watchdog +
barra de progresso em modo Reunião. - **Bug corrigido em uso real: modo Reunião
não tinha barra de progresso nem proteção contra travamento; placeholder "processando..."
ficava preso indefinidamente se o backend travasse ou a API retornasse overloaded. -
`useReuniao.ts` ganha `agentProgress`, `agentProgressMeta`, `progressIntervalsRef`,
`watchdogTimersRef`, `activeAgentsRef`, `timedOutAgentsRef`. Padrão T90 replicado:
`agent\_start` → `fetch mediana` → `setInterval(200ms)` + watchdog; `agent\_done` → `snap 100%`
+ clear. Watchdog = `max(60, min(mediana - 3, 1200))` - 1000 ms (cap 20 min).
`timedOutAgentsRef` evita ghost bubbles pós-watchdog. - **`useChat.ts` (Pipeline) recebe o
mesmo watchdog por consistência: `watchdogTimersRef` + `timedOutAgentsRef` + guards em
`token` / `agent\_done`. Limpo em `abort()`, `reset()`, `ws.onclose`. - **`ChatPanel.tsx`: aceita
`reunAgentProgress?` e `reunAgentProgressMeta?`; `showBar` usa a fonte correta por modo
(pipeline vs reunião) - ProgressBar agora renderiza em ambos os modos. MacroBar não
aparece em Reunião. - **`page.tsx`: passa `reunAgentProgress` / `reunAgentProgressMeta` ao
ChatPanel. - **Manual: 3.2 atualizado (watchdog + barra de progresso), 4.17 adicionado.
--- ### v1.24 - 2026-05-07 **FASE 6 - T93: export agente-agnóstico (Renata sem export
próprio). - **Bug corrigido em uso real: `POST /exportar` era hardcoded para
`respostas.aya`; output da Renata ficava vazio (gerado, salvo no JSON, exibido no chat mas
sem caminho de export). - **`ExportarPayload` ganha `agente: str = "aya"` default
mantém compatibilidade total com chamadas antigas. - **`/exportar` usa `payload.agente`
para extrair markdown, definir `out\_dir = OUTPUTS\_DIR / agente` (com `mkdir`), e montar
mensagem de erro descritiva. `agentes\_detectados` passado como `[]` para agentes != "aya". -
`/download/{session\_id}/{tipo}` aceita `?agente=X` (Query param, default "aya") para
servir o arquivo do subdiretório correto. - **Frontend `ChatPanel`: dois botões
independentes ("Exportar Dossiê (Aya)" / "Exportar Editorial (Renata)"), cada um com estado
próprio (`exportStates` / `exportResults` como `Record`). Aparecem somente quando o agente
correspondente rodou. Download inclui `?agente=X`. - **Manual 4.16. --- ### v1.23 -
2026-05-07 **BLOCO 5 - T80 + T86: gráfico de latência e toast global. - **T80 -
`GET /saude/latencias?agente=X&dias=30`: mÃdias semanais de `duracoes\_segundos` por
agente. Semanas com mÃdia > 120s retornam `lenta: true`. Registrado em `api/routes/
saude.py`. - \*\*T80 - recharts line chart em `/saude`: seÃs "Latência Semanal" com
selector de agente e período (30/60/90 dias). Pontos e linha mudam para vermelho em semanas
lentas. Linha de referência em 120s. - **T86 - sonner + `@/lib/toast`: `Toaster` em
`layout.tsx` (bottom-right, rich colors). Wrapper `notify.{error,success,info,warning}` em `lib/
toast.ts`. `useApiQuery` agora dispara `notify.error()` automaticamente em falhas. /briefing-

reverso` e `/cortes` refatorados para remover estado `error` inline. - \*\*Manual:\*\*` Â§4.14 (gráfico latência), Â§4.15 (toast global). --- #### v1.21` 2026-05-07 \*\*BLOCO 5` T83 + T87 + T84: segurança de arquivo e scripts de manutenção.` - \*\*T83` ``chmod 600 .env``:`` permissão do arquivo .env` restrita ao dono (-rw-----). Script `scripts/setup\_seguro.sh` aplica `600` no .env` e `700` em `backups/` `` executar após clonar o repositório. Documentado em Â§8.5. - **T87` ``scripts/limpar\_outputs.py``:`` limpeza de outputs por idade (--dias, padrão 30). Lê `historico/dashboard/\*.json` diretamente para identificar sessões 5 e preservá-las. Flag `--dry-run` obrigatória para simulação. Mínimo de 7 dias recusado automaticamente. Documentado em Â§5.11. - \*\*T84` ``scripts/backup_historico.py``:`` backup compactado (ZIP deflate) de `historico/`, `outputs/`, `inputs/clientes/` e `core/exemplares/`. Destino padrão `backups/lemmon-YYYYMMDD\_HHMMSS.zip`. Aceita --destino` para HD externo. Documentado em Â§5.12. --- #### v1.20` 2026-05-07 **BLOCO HIGIENE` T91: auditoria completa Â§2` Â§8. - **Â§2 "A equipe":`` corrigido para 7 agentes (Renata era o 7º mas o título/intro ainda dizia 6). Parágrafo de introdução atualizado para mencionar Renata como opcional no pipeline. - \*\*Â§3.1 Pipeline:`` sequência atualizada para "Otto` Heitor` Salles` Sônia` Aya (Aya` Renata, opcional)" `` reflete toggle editorial do ConfigSidebar. - \*\*Â§4 numeração:`` Â§4.10 Escritório virtual renumerado para Â§4.13 (aparecia fora de ordem depois de Â§4.11 e Â§4.12 adicionados mais tarde). - **Â§4.13 Escritório virtual:`` descrição expandida para incluir whiteboard ao vivo (v1.7) e mic destacado em reunião (v1.7) `` estava incompleto e sem mencionar features já entregues. - **Â§5:`` nova seção Â§5.7 Renata CLI. Seções Â§5.7` Â§5.9 renumeradas para Â§5.8` Â§5.10. - \*\*Â§7 Roadmap:`` reescrita completa. Tabelas de concluídos por Cópico (FASE 1`4) e por bloco (FASE 5). Pendentes: BLOCO HIGIENE + BLOCO 5. Substituiu texto completamente fora de data ("v1.7, 37 tarefas, todos Cópicos entregues"). - **Â§8.1 Custos:`` adicionada linha Renata (\$0.10`0.25). Custo total pipeline com/sem Renata documentado. - \*\*Â§8.4 Estrutura de pastas:`` expandida para refletir estrutura atual: `api/` modularizado, `core/tipos.py`, `core/calendario\_br.py`, `core/templates/`, `outputs/renata/estoque/`, `historico/dashboard/`, estrutura completa do `dashboard/`. Comentário `agentes/` corrigido para 7 agentes. - **Â§9.2 Como atualizar:`` passo "Atualizar CHANGELOG.md" removido como obrigatório `` T92 torna o manual a fonte primária; nota explicativa adicionada. --- #### v1.19` 2026-05-07 \*\*BLOCO 6` T89: tema claro/escuro.` - \*\*`next-themes``:`` `ThemeProvider` em `layout.tsx`. `darkMode: 'class'` em `tailwind.config.js`. Preferência persiste em localStorage (`lemmon-theme`); default: `prefers-color-scheme` do SO. - **Toggle` no header ao lado do botão de histórico. - \*\*`dark:`` classes`` cobrindo: `body` + `.glass` (via globals.css), `.iso-tile`, header e logo, agent pills, botões de navegação, ChatPanel (header, content, textarea), MacroBar, ProgressBar. - **Cores dos agentes:`` mantidas (`colorDim` pastéis funcionam em dark). Aya #18181b não inverte `` bolha near-white em fundo escuro tem contraste adequado. - **Exporta``:`` HTML/PDF da Aya não afetados (documento de impressão). --- #### v1.18` 2026-05-07 \*\*BLOCO 6` T90 V2: barra de progresso + ETA por agente.` - \*\*Backend `GET /sessoes/medianas?agente=X``:`` lê últimas 20 sessões de `historico/dashboard/`, calcula mediana de `duracoes\_segundos[agente]`. Cache in-memory TTL 60s. Retorna `{mediana\_segundos, amostras}` ou `null` se < 3 amostras. - **`useChat.ts` `` progress state:`` `agentProgress` (0`100) e `agentProgressMeta` (mediana/elapsed/amostras) por agente. `setInterval(200ms)` em `agent_start`, snap 100% em `agent_done`. Cleanup em `abort()`, `ws.onclose`, e `useEffect` unmount `` sem memory leak. - \*\*FALLBACK\_MEDIANAS:`` `{otto:20, heitor:40, salles:30, sonia:30, aya:15, pedro\_abrahao:25, renata:30}` em segundos, usado quando endpoint retorna null. - **Salles A/B:`` quando `agentConfig.salles.alternativas === 3`, mediana — 3 antes de calcular progresso (frontend, ambos os caminhos fetch e catch). - \*\*`ProgressBar.tsx``:`` micro barra abaixo de cada bubble. Cor do agente, animação 200ms. Tooltip: "Tempo médio: Xs. decorrido: Ys. n=Z amostras". Cap 95% antes de `agent\_done`; snap 100% em `agent\_done`. - \*\*Overloaded:`` `elapsed > mediana \* 1.5` + ainda não concluído `barra` + texto "Mais lento que o normal". - **Manual lock:`` após `agent\_done` em modo manual, barra trava em 100% com "Aguardando aprovação..." até operador

decidir. - **MacroBar.tsx**: visÃo geral do pipeline no header. Ãcones â±/â±/â± por agente + mini barra para agentes ativos. - **DocumentaÃo**: 3.3 e 4.11 atualizados. --- **v1.17** 2026-05-07 **FASE 5** BLOCO 4: doc + versionamento de materiais espelho (T76 â† T77 â† T85). - **T76** Receitas 6 (7 novas): 6.7 Modo Remix, 6.8 Fast-track, 6.9 Sandbox, 6.10 A/B Salles, 6.11 Briefing Reverso, 6.12 Cortes-prontos, 6.13 Gate Espelho. Cada receita com 4 "6 passos, referenciando a feature real no cÃdigo. - **T77** 10 "Como adicionar cliente espelho": Guia de 8 etapas do onboarding: `onboard\_cliente.py`, dossiÃ, transcriÃÃes, system prompt, instanciaÃo Python, snippet TS em `agents.ts`, alias em `AGENTE\_ALIAS`, teste CLI. - **T85** Versionamento de materiais primÃrios (cÃdigo): `EspelhoCliente.\_carregar\_material\_primario()` calcula SHA-256 (primeiros 12 chars) do material combinado e registra em `self.\_material\_hash`. Campo `material\_hash` adicionado ao resultado de cada execuÃo (`AgenteResultado` TypedDict atualizado). SessÃes antigas sem o campo tratam com `?? "?"`. Pasta `inputs/clientes/pedro/historico/` criada como convenÃo de arquivo de versÃes antigas. --- **v1.16** 2026-05-07 **Renata** Social Media (T20): linha editorial Instagram com narrativa conectada. - **Renata**: SÃtimo agente do sistema. Recebe o dossiÃ da Aya e produz linha editorial Instagram de n dias (1 post/dia), com storytelling condicional, datas comemorativas do nicho do cliente e CTA por peÃsa. Apenas 3 formatos: Reels, Carrossel, Stories. Output humano limitado a 5000 chars. Material tÃcnico inclui `linha\_narrativa`, `publicacoes`, `descartes`, `estatisticas\_mix`. - **core/calendario_br.py**: Banco de datas comemorativas BR filtradas por nicho. Inclui `DATAS\_FIXAS` (54 datas), `DATAS\_MOVEIS` (2026â€2027), `CAMPANHAS\_MES\_INTEIRO` (Outubro Rosa, Novembro Azul, etc.) e `datas\_na\_janela(inicio, fim, nichos)`. - **Modos de operaÃo**: `pipeline` (recebe dossie_aya + roteiro_salles + analise_sonia + diretrizes_heitor) e `solo` (contexto livre â€ retorna 3 perguntas se contexto raso, gera direto se contexto rico). - **Descartes**: Material nÃo aproveitado salvo em `outputs/renata/estoque/\_descartes.txt` para reuso posterior. - **ConfigSidebar**: Toggle "editorial (~\$0.20)" + range de duraÃo 1â€60 dias. Dispara via `config.renata.{incluir, duracao\_dias}` no payload WS. - **Frontend completo**: sprite SVG (roupa coral #e11d48, prancheta com post-its coloridos), ROLES, IDLE_QUOTES, posiÃÃes no escritÃrio. - **nichos_calendario** no EspelhoCliente: campo opcional `list[str]` para filtrar datas BR relevantes ao nicho do cliente. Configurado como `nacional, saude, mulher, medico` no dossiÃ de Pedro. - **8 smoke tests pytest** (padrÃo T67) cobrindo: datas_na_janela, pipeline, modo solo raso, validaÃÃes de input. --- **v1.15** 2026-05-07 **FASE 5** BLOCO 3: tipagem, hooks TS e refatoraÃo de agentes (T74 â† T75 â† T82 â† T81 â† T73). - **T74** api-client.ts centralizado: `dashboard/lib/api-client.ts` criado com `apiFetch` utilitÃrio Ãnico e 7 funÃÃes tipadas exportadas (`fetchBriefingReverso`, `fetchCortesProntos`, `fetchHistorico`, `fetchCalibragemPedro`, `postCalibragemPedro`, `fetchShare`, `postComentario`). As 6 pÃginas do dashboard refatoradas para usar o client (eliminados ~80 `fetch()` inline repetidos). - **T75** useApiQuery hook: `dashboard/lib/use-api-query.ts` exporta `useApiQuery(fn, deps)` genÃrico retornando `{ data, loading, error, reload }`. Padroniza o ciclo de loading/error em todas as pÃginas que consultam API. - **T82** HistoryPanel splitado: `dashboard/components/history/HistoryPanel.tsx` de 582 linhas dividido em 4 arquivos: `FilterBar.tsx` (filtros de perÃodo/origem), `SessionCard.tsx` (card de sessÃo), `SessionDetail.tsx` (painel de detalhes expandido), `HistoryPanel.tsx` reduzido para 146 linhas como orquestrador. - **T81** SÃnia migra para _chamar_api da base: As 3 chamadas diretas a `self.client.messages.create()` em `agentes/sonia.py` migradas para `self.\_chamar\_api()`. Imports de `APIError`, `AuthenticationError`, `RateLimitError` e `Custo` removidos do arquivo. Teste diferencial confirmou output idÃntico ao baseline prÃo-refatoraÃo. - **T73** AgenteResultado TypedDict + annotations: `core/tipos.py` criado com `AgenteResultado(TypedDict, total=False)` contendo 4 campos base garantidos (`output\_humano`, `output\_tecnico`, `custo\_total\_usd`, `duracao\_segundos`) mais campos extras comuns opcionais. Todos os agentes e `EspelhoCliente` anotam `executar() -> AgenteResultado`. `AgenteBase` atualiza tipo de retorno abstract. Returns com campos extras usam `cast()`. `chamar\_api chain` e `somar custo` documentados com distinÃo explÃcita: chain para

chamadas independentes (mesma entrada), somar+_chamar_api individual para chamadas dependentes (output N-1 alimenta N). --- #### v1.14 â€” 2026-05-06 **FASE 5 â€” BLOCO 1: refatoraÃ§Ã£o de dÃ¡vida tÃ©cnica (T61 â†’ T62 â†’ T63 â†’ T64 â†’ T65 â†’ T78).** - **T61 â€” Modularizar api_server.py:** MonÃ³lito de ~1317 linhas quebrado em `api/` com 10 mÃ³dulos: `main.py` (FastAPI + middlewares), `deps.py` (globals compartilhados), `schemas.py` (modelos Pydantic), `storage.py` (persistÃªncia de sessÃµes), `ws\_helpers.py` (utilidades WebSocket), `ws\_chat.py`, `ws\_reuniao.py`, `ws\_mesa.py` e routers por domÃnio (`historico`, `exportar`, `exemplares`, `auxiliares`, `transcrever`, `share`, `calibragem`). `api\_server.py` vira shim de 1 linha. Output idÃ©ntico. - **T62 â€” print() â†’ logger:** SubstituÃdos todos os `print()` restantes em `agentes/aya.py`, `agentes/heitor.py`, `agentes/sonia.py` e `core/espelho.py` por `self.logger.info()` / `self.logger.warning()`. Duplicatas (print + logger.warning no mesmo bloco) eliminadas. Output dos agentes inalterado. - **T63 â€” Extrair CSS do exportador:** String inline de ~455 linhas do AURA Design System extraÃda de `core/exportador\_aya.py` para `core/templates/aura.css`. Leitura via `Path(\_\_file\_\_).parent / "templates" / "aura.css"` em tempo de importaÃ§Ã£o. CSS byte-idÃ©ntico ao anterior. - **T64 â€” Splitar OfficeScene.tsx:** Componente SVG isomÃ©trico de 1627 linhas dividido em 5 arquivos em `dashboard/components/office/`: `constants.ts` (temas, coordenadas, ROLES, IDLE_QUOTES), `SpeechBubble.tsx`, `WorkRoom.tsx` (estÃ¡dio + todos os mÃ³veis), `MeetingRoom.tsx` e `ReceptionRoom.tsx`. `OfficeScene.tsx` reduzido para 594 linhas com apenas posiÃ§Ãµes de personagens, estado de movimento e componente principal. TypeScript sem erros. - **T65 â€” Splitar ChatPanel.tsx:** Componente de 1493 linhas reorganizado: `MessageBubble.tsx` recebe `exportTxt`, `UserMessage` e `AgentMessage`; `ConfigSidebar.tsx` recebe o sidebar de configuraÃ§Ãµes com interface prÃ³pria (sem `Props['onUpdateConfig']`). `ChatPanel.tsx` importa dos novos mÃ³dulos e reduz para 1247 linhas. TypeScript sem erros. - **T78 â€” Heitor usa _chamar_api da base:** As 3 chamadas diretas a `self.client.messages.create()` em `agentes/heitor.py` foram migradas para `self.\_chamar\_api()` da base, que centraliza exception handling (`formatar\_erro\_anthropic`), timing e cÃ¡lculo de custo. Imports de `APIError`, `AuthenticationError`, `RateLimitError` e `Custo` removidos do arquivo. Output do agente inalterado. --- #### v1.13 â€” 2026-05-06 **FASE 4 â€” T60: polimento UI (3 fixes).** - **Barra "Uso de agentes" sem rÃ3tulo (T60-1):** Dashboard `/saude` usava `agent.color` como cor do texto â€” Aya tem `#18181b` (quase preto) invisÃvel sobre fundo escuro. Corrigido: label usa `text-stone-300` consistente, com bolinha colorida Ã esquerda para manter identidade visual do agente. Todos os labels visÃveis em todos os temas. - **Speech bubble "DossiÃa pronto!" sem affordance de clique (T60-2):** BalÃ£o de fala era visualmente idÃ©ntico ao decorativo mas aparecia em zona clicÃvel do agente. Adicionado `cursor: pointer` explÃcito ao balÃ£o e hover state via CSS (`speech-bubble-group:hover .speech-bubble-rect` â†’ stroke mais espesso + drop-shadow). O clique jÃ funcionava (toggleAgent) â€” agora Ã© visualmente Ãbvio. - **Outros achados visuais (T60-3):** Sem achados adicionais registrados nesta rodada de testes. --- #### v1.12 â€” 2026-05-06 **FASE 4 â€” T56+T57+T58+T59: QA, TTS, share e telemetria.** - **data-testid no Whiteboard (T56):** `` adicionado ao container do componente SVG. QA pode usar `document.querySelector('[data-testid=whiteboard]')` ao invÃ©s de `[class\*=whiteboard]` que falha em elementos ``. ConfirmaÃ§Ã£o visual de preenchimento das barras depende de rodar pipeline no Chrome. - **TTS robusto com detecÃ§Ã£o de voz pt-BR (T57):** `window.speechSynthesis.speak()` silenciava sem erro quando sem voz pt-BR instalada. Reescrito: enumera vozes via `getVoices()`, detecta pt-BR ou pt por `lang.startsWith`, reporta erro no estado (`ttsError`) se nenhuma encontrada. Ãcone altera â€”/â€” (amber quando falando). Console.log das vozes disponÃveis facilita diagnÃstico em novos ambientes. - **PÃgina `/share/[token]` lÃ¡ `err.detail` (T58):** Erros da API agora exibem mensagem legÃvel em vez de "Falha na anÃlise" genÃrica, consistente com T52. - **Telemetria de latÃªncia por agente (T59):** Cada execuÃ§Ã£o de agente no WebSocket pipeline agora registra duraÃ§Ã£o em `duracoes[name]`. `\_salvar\_sessao` persiste `"duracoes\_segundos": {...}` no JSON da sessÃ£o. `HistoryPanel` exibe `â€± Xs` ao lado do custo de cada agente (amber quando >120s). SessÃµes antigas sem `duracoes\_segundos` sÃ£o compatÃveis via guard `?? {}`. --- #### v1.11 â€” 2026-05-06 **FASE 4 â€” T51+T54+T55: menÃ§Ãµes, URL e tags.** - **Alias

`@pedro` em reunião (T51):** `\_parse\_mentions` agora usa `AGENTE\_ALIAS` dict para casar nome curto com id composto. `@pedro` aciona Pedro Abrahão; `@pedro\_abrahao` (id completo) também funciona. Adicionar aliases novos em `AGENTE\_ALIAS` ao onboar novos clientes espelho. - **URL `/cortes` verificada (T54):** Rota existe, build confirma HTTP 200, link do header `i` abre a página sem 404. O 404 visto no teste E2E era de estado anterior. Verificado em 2026-05-06. - **Tags com log e fallback heurístico (T55):** Bloco de tags substitui `except: pass` por `\_log.warning` com tipo e mensagem (visível nos logs do uvicorn). Fallback heurístico: quando Haiku falha, extrai 3 palavras mais frequentes do briefing (filtradas por stopwords pt-BR) e envia como chips com prefixo `auto:`. Evento `tags\_sugeridas\_falhou` enviado para rastreamento. Feature degrada graciosamente: operador sempre recebe algum sinal. --- ### v1.10 2026-05-06 **FASE 4** T52+T53: erros Anthropic legíveis (bloco crítico sistêmico).** - **Helper `formatar\_erro\_anthropic` (T53):** `core/agente\_base.py` centraliza a formatação de erros do SDK Anthropic. `\_chamar\_api` e `\_chamar\_api\_stream` usam tipos específicos (`APIError`, `APIConnectionError`, `AuthenticationError`, `RateLimitError`) sem `Exception` genérico que mascararia bugs prioritários. Mensagens pt-BR claras para overloaded, rate_limit, auth inválida e sem conexão. Chat nunca exibe `{'type': 'error', ...}` como string Python. - **Endpoints LLM protegidos (T52):** `/sugerir\_pipeline`, `/briefing\_reverso` e `/cortes\_prontos` agora envolvem a chamada Anthropic com os mesmos tipos específicos `HTTPException(503, detail=)`. Frontend de cada página lê `err.detail` e exibe a mensagem no estado de erro (antes mostrava "Failed to fetch" ou string genérica). --- ### v1.9 2026-05-06 **FASE 3** T45 a T50: robustez, UX e polish.** - **Piso no autorizar_custo (T45):** Valor 0 ou negativo no payload `autorizar\_custo` agora é clampeado para mínimo \$0.10, evitando loop infinito de "cap atingido". - **Risco vermelho Heitor via campo estruturado (T46):** Detecção de `risco\_geral` do `output\_tecnico` (dict) antes de fazer fallback no emoji ðŸ”’ é robusto a mudanças de formatação do modelo. - **Autocomplete session_id na calibragem (T47):** Página `/calibragem` busca `/historico` ao montar e alimenta`